

Lesson 12: Hybrid Quantum-Classical Neural Networks

Lesson 12: Hybrid Quantum-Classical Neural Networks EstimatorQNN, TorchConnector, PyTorch training, and interpretation

Dr. Syed Muzamil Basha

School of Computer Science and Engineering, REVA University

Learning Outcomes

- Explain a hybrid quantum-classical neural network.
- Identify classical and quantum layers in the model.
- Understand how PyTorch backpropagation interacts with a quantum layer.
- Interpret the training-loss and test-accuracy outputs.

What Makes This Model Hybrid?

- The input and output layers are classical PyTorch layers.
- The middle layer is a quantum neural network.
- PyTorch manages the overall optimization process.
- The quantum circuit contributes a trainable nonlinear transformation.

Classical FC $\rightarrow$ Quantum Layer $\rightarrow$ Classical FC

- `fc1` maps two inputs to two values.
- `TorchConnector` makes the quantum network act like a PyTorch layer.
- `fc2` maps the quantum output to two class scores.

Creating the Quantum Layer

```
X, y = make_moons(n_samples=100, noise=0.1, random_state=42)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42)
num_qubits = 2
quantum_circuit = RealAmplitudes(num_qubits, reps=3)
estimator = StatevectorEstimator()
qnn = EstimatorQNN(circuit=quantum_circuit, estimator=estimator,
                    input_params=quantum_circuit.parameters[:2],
                    weight_params=quantum_circuit.parameters[2:])
quantum_layer = TorchConnector(qnn)
class HybridQuantumNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(2, 2)
        self.quantum = quantum_layer
        self.fc2 = nn.Linear(1, 2)
    def forward(self, x):
        x = torch.tanh(self.fc1(x))
        x = self.quantum(x)
```

Training: Same ML Loop, New Middle Layer

- The classical optimizer is Adam.
- Loss is cross-entropy.
- `loss.backward()` computes gradients through the hybrid model.
- The optimizer updates both classical and quantum parameters exposed to PyTorch.

Interpreting the Recorded Output

Epoch 10: 0.7193 Epoch 20: 0.6958 Epoch 50: 0.6861

Test Accuracy: 30.00%

- Loss decreases gradually in the recorded run.
- The final test accuracy is low for this dataset: 30%.
- This illustrates that a hybrid architecture can train without necessarily producing a strong classifier on a small example.

Important Beginner Lesson

- A quantum circuit inside a neural network does not automatically create quantum advantage.
- Architecture, initialization, optimizer, dataset, encoding and noise all influence performance.
- The goal here is to understand the integration pattern.

- Quantum circuits can be exposed as differentiable model components.
- PyTorch can orchestrate the full model.
- The notebook records 30% test accuracy.

Hybrid quantum-classical learning combines familiar neural-network training with a quantum computational layer.

Quick Check

- ① Which layers are classical?
- ② Which layer is quantum?
- ③ What do input parameters and weight parameters mean?
- ④ Does a quantum layer guarantee better accuracy?